

课程编号： IB00000

得分	教师签名	批改日期



深圳技术大学课程设计报告

课程名称： 程序设计基础 B

班 级：

指导教师：

报 告 人： 邹宇恒 学号：

实验地点： 深圳技术大学

实验时间： 2025 年 6 月 15 日

提交时间： 2025 年 6 月 17 日

一、课程设计题目和要求

在提供的数据集中选择 **Super Market Sales**，对商场销售数据进行统计分析与数据可视化（由于数据完整清洗，无进行数据清洗的必要），使用 **python** 及 **AI** 辅助，完成从统计清洗到商业建议的全流程数据分析任务，并输出可视化报告（PPT 形式）。

二、课程设计实现思路

通过 **python** 及 **AI** 辅助下，不断引导 **AI** 逐步完善代码内容。先让 **AI** 将 **CSV** 数据导入 **spyder** 环境，并输出 5 行数据检查数据是否正常录入。检查数据录入成功后，进行一系列分析，包括：销售趋势分析，客户类型分析，产品线表现分析，支付方式偏好分析，分店销售分析，城市销售分析，销售时间分析，然后再拟合 **ARIMA** 模型进行未来 5 天的销售额的预测，绘制 **ARIMA** 模型预测结果并保存所有图表。

最后，以用户满意度分析为出发点，分别分析了按客户类型分组的评分分布，按支付方式分组的评分分布，评分与客户满意度的关系，评分随时间的变化趋势，绘制数据可视化结果并保存所有图表。

在得到图表的基础上，分析其含义，并相应给出商业建议，制作成商业报告。

三、课程设计实现过程与代码

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
Created on Tue Jun 10 15:29:07 2025
```

```
@author: xingh
```

```
"""
```

```
#由于本数据集完整，无进行数据清洗和数据预处理的必要
```

```
# 导入必要的库
```

```
import pandas as pd # 用于数据处理和分析
```

```
import matplotlib.pyplot as plt # 用于数据可视化
```

```
import seaborn as sns # 用于数据可视化（基于 matplotlib，提供更美观的样式）
```

```
from sklearn.model_selection import train_test_split # 用于数据集划分
```

```
from sklearn.preprocessing import StandardScaler # 用于特征标准化
```

```
from sklearn.linear_model import LinearRegression # 用于线性回归模型
```

```
from sklearn.metrics import mean_squared_error, r2_score # 用于模型评估指标
```

```
import statsmodels.api as sm # 用于统计分析和时间序列分析
```

```
from statsmodels.tsa.arima.model import ARIMA # 用于 ARIMA 时间序列模型
```

```
import warnings # 用于警告处理
```

```
import os # 用于文件路径操作
```

```
# 创建输出文件夹（如果不存在）
```

```
output_folder = "output_charts"
```

```
os.makedirs(output_folder, exist_ok=True)
```

```

# 读取 CSV 文件（使用原始字符串处理路径）
file_path = r"D:\pro design project\result\supermarket_sales.csv"
supermarket_data = pd.read_csv(file_path)

# 显示数据前 5 行
print("数据导入成功！前 5 行数据预览：")
print(supermarket_data.head())

# 数据分析与可视化部分

# 1. 销售趋势分析
# 将 'Date' 列转换为日期格式
supermarket_data['Date'] = pd.to_datetime(supermarket_data['Date'])

# 按日期统计每日销售额
daily_sales = supermarket_data.groupby('Date')['Total'].sum().reset_index()

# 绘制每日销售趋势图
plt.figure(figsize=(10, 6))
sns.lineplot(x='Date', y='Total', data=daily_sales, marker='o')
plt.title('Daily Sales Trend') # 设置标题
plt.xlabel('Date') # 设置 x 轴标签
plt.ylabel('Total Sales') # 设置 y 轴标签
plt.grid(True) # 添加网格

# 保存图表
plt.savefig(os.path.join(output_folder, 'daily_sales_trend.png'), dpi=300, bbox_inches='tight')
plt.show()

# 2. 客户类型分析
# 统计会员和普通客户的购买次数和销售额
customer_type_analysis = supermarket_data.groupby('Customer type').agg({
    'Invoice ID': 'count', # 统计购买次数
    'Total': 'sum' # 统计销售额
}).rename(columns={'Invoice ID': 'Purchase Count', 'Total': 'Total Sales'}).reset_index()

# 绘制客户类型销售对比图
plt.figure(figsize=(8, 5))
sns.barplot(x='Customer type', y='Total Sales', data=customer_type_analysis, palette='viridis')
plt.title('Sales by Customer Type')
plt.xlabel('Customer Type')
plt.ylabel('Total Sales')
plt.grid(axis='y') # 添加 y 轴网格

```

```

# 保存图表
plt.savefig(os.path.join(output_folder, 'customer_type_sales.png'), dpi=300,
bbox_inches='tight')
plt.show()

# 3. 产品线表现分析
# 统计各产品线的销售额和销售量
product_line_analysis = supermarket_data.groupby('Product line').agg({
    'Quantity': 'sum', # 统计销售量
    'Total': 'sum' # 统计销售额
}).rename(columns={'Quantity': 'Total Quantity', 'Total': 'Total Sales'}).reset_index()

# 绘制产品线销售分析图
plt.figure(figsize=(10, 6))
sns.barplot(x='Product line', y='Total Sales', data=product_line_analysis, palette='rocket')
plt.title('Sales by Product Line')
plt.xlabel('Product Line')
plt.ylabel('Total Sales')
plt.xticks(rotation=45) # 旋转 x 轴刻度标签
plt.grid(axis='y')

# 保存图表
plt.savefig(os.path.join(output_folder, 'product_line_sales.png'), dpi=300, bbox_inches='tight')
plt.show()

# 4. 支付方式偏好分析
# 统计不同支付方式的使用次数和金额
payment_analysis = supermarket_data.groupby('Payment').agg({
    'Invoice ID': 'count', # 统计交易次数
    'Total': 'sum' # 统计交易金额
}).rename(columns={'Invoice ID': 'Transaction Count', 'Total': 'Total Amount'}).reset_index()

# 绘制支付方式分析图
plt.figure(figsize=(8, 5))
sns.barplot(x='Payment', y='Total Amount', data=payment_analysis, palette='pastel')
plt.title('Sales by Payment Method')
plt.xlabel('Payment Method')
plt.ylabel('Total Amount')
plt.grid(axis='y')

# 保存图表
plt.savefig(os.path.join(output_folder, 'payment_method_sales.png'), dpi=300,
bbox_inches='tight')

```

```

plt.show()

# 5. 分店销售分析
# 统计各分店的销售额
branch_sales = supermarket_data.groupby('Branch')['Total'].sum().reset_index()

# 绘制分店销售分析图
plt.figure(figsize=(8, 5))
sns.barplot(x='Branch', y='Total', data=branch_sales, palette='Set2')
plt.title('Sales by Branch')
plt.xlabel('Branch')
plt.ylabel('Total Sales')
plt.grid(axis='y')

# 保存图表
plt.savefig(os.path.join(output_folder, 'branch_sales.png'), dpi=300, bbox_inches='tight')
plt.show()

# 6. 城市销售分析
# 统计各城市的销售额
city_sales = supermarket_data.groupby('City')['Total'].sum().reset_index()

# 绘制城市销售分析图
plt.figure(figsize=(8, 5))
sns.barplot(x='City', y='Total', data=city_sales, palette='Set1')
plt.title('Sales by City')
plt.xlabel('City')
plt.ylabel('Total Sales')
plt.grid(axis='y')

# 保存图表
plt.savefig(os.path.join(output_folder, 'city_sales.png'), dpi=300, bbox_inches='tight')
plt.show()

# 7. 销售时间分析
# 提取小时信息
supermarket_data['Hour'] = pd.to_datetime(supermarket_data['Time']).dt.hour

# 按小时统计销售额
hourly_sales = supermarket_data.groupby('Hour')['Total'].sum().reset_index()

# 绘制每小时销售趋势图
plt.figure(figsize=(10, 6))
sns.lineplot(x='Hour', y='Total', data=hourly_sales, marker='o')

```

```

plt.title('Hourly Sales Trend')
plt.xlabel('Hour')
plt.ylabel('Total Sales')
plt.grid(True)

# 保存图表
plt.savefig(os.path.join(output_folder, 'hourly_sales_trend.png'), dpi=300, bbox_inches='tight')
plt.show()

# 时间序列模型：ARIMA 模型预测
# 将数据按日期排序并设置日期为索引
daily_sales = daily_sales.sort_values('Date')
daily_sales.set_index('Date', inplace=True)

# 拟合 ARIMA 模型
warnings.filterwarnings("ignore") # 忽略收敛警告
arima_model = ARIMA(daily_sales['Total'], order=(5, 1, 0)) # 这里的 order 参数需要根据数
据进行调整
arima_result = arima_model.fit()

# 预测未来 5 天的销售额
forecast = arima_result.forecast(steps=5)
print("\nARIMA 模型预测未来 5 天的销售额:")
print(forecast)

# 绘制 ARIMA 模型预测结果
plt.figure(figsize=(10, 6))
plt.plot(daily_sales['Total'], label='Historical Sales') # 绘制历史销售数据
plt.plot(pd.date_range(start=daily_sales.index[-1], periods=6, freq='D')[1:], forecast,
label='Forecasted Sales', color='red') # 绘制预测数据
plt.title('ARIMA Sales Forecast')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.legend()
plt.grid(True)

# 保存图表
plt.savefig(os.path.join(output_folder, 'arima_sales_forecast.png'), dpi=300,
bbox_inches='tight')
plt.show()

# 用户满意度分析与可视化

# 评分分布分析

```

```

plt.figure(figsize=(10, 6))
sns.histplot(supermarket_data['Rating'], bins=10, kde=True)
plt.title('Distribution of Customer Ratings')
plt.xlabel('Rating')
plt.ylabel('Frequency')
plt.grid(True)
plt.savefig(os.path.join(output_folder, 'rating_distribution.png'), dpi=300, bbox_inches='tight')
plt.show()

# 按客户类型分组的评分分布
plt.figure(figsize=(10, 6))
sns.histplot(data=supermarket_data, x='Rating', hue='Customer type', element='step',
stat='density', common_norm=False)
plt.title('Rating Distribution by Customer Type')
plt.xlabel('Rating')
plt.ylabel('Density')
plt.grid(True)
plt.savefig(os.path.join(output_folder, 'rating_by_customer_type.png'), dpi=300,
bbox_inches='tight')
plt.show()

# 按支付方式分组的评分分布
plt.figure(figsize=(10, 6))
sns.histplot(data=supermarket_data, x='Rating', hue='Payment', element='step', stat='density',
common_norm=False)
plt.title('Rating Distribution by Payment Method')
plt.xlabel('Rating')
plt.ylabel('Density')
plt.grid(True)
plt.savefig(os.path.join(output_folder, 'rating_by_payment.png'), dpi=300, bbox_inches='tight')
plt.show()

# 评分与客户满意度的关系
plt.figure(figsize=(10, 6))
sns.boxplot(x='Customer type', y='Rating', data=supermarket_data)
plt.title('Customer Satisfaction by Customer Type')
plt.xlabel('Customer Type')
plt.ylabel('Rating')
plt.grid(True)
plt.savefig(os.path.join(output_folder, 'satisfaction_by_customer_type.png'), dpi=300,
bbox_inches='tight')
plt.show()

plt.figure(figsize=(10, 6))

```

```

sns.boxplot(x='Payment', y='Rating', data=supermarket_data)
plt.title('Customer Satisfaction by Payment Method')
plt.xlabel('Payment Method')
plt.ylabel('Rating')
plt.grid(True)
plt.savefig(os.path.join(output_folder, 'satisfaction_by_payment.png'), dpi=300,
bbox_inches='tight')
plt.show()

# 评分随时间的变化趋势
supermarket_data['Date'] = pd.to_datetime(supermarket_data['Date'])
rating_over_time = supermarket_data.groupby('Date')['Rating'].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.lineplot(x='Date', y='Rating', data=rating_over_time, marker='o')
plt.title('Customer Satisfaction Trend Over Time')
plt.xlabel('Date')
plt.ylabel('Average Rating')
plt.grid(True)
plt.savefig(os.path.join(output_folder, 'rating_trend.png'), dpi=300, bbox_inches='tight')
plt.show()

```

四、程序运行结果

```

In [1]: runfile('D:/pro design project/result/20250615-zouyuheng.py', wdir='D:/pro design project/result')
数据导入成功！前5行数据预览：
   Invoice ID Branch  City  ... gross margin percentage gross income Rating
0  750-67-8428     A   Yangon  ...      4.761905      26.1415      9.1
1  226-31-3081     C  Naypyitaw  ...      4.761905       3.8200      9.6
2  631-41-3108     A   Yangon  ...      4.761905      16.2155      7.4
3  123-19-1176     A   Yangon  ...      4.761905      23.2880      8.4
4  373-73-7910     A   Yangon  ...      4.761905      30.2085      5.3

[5 rows x 17 columns]

```

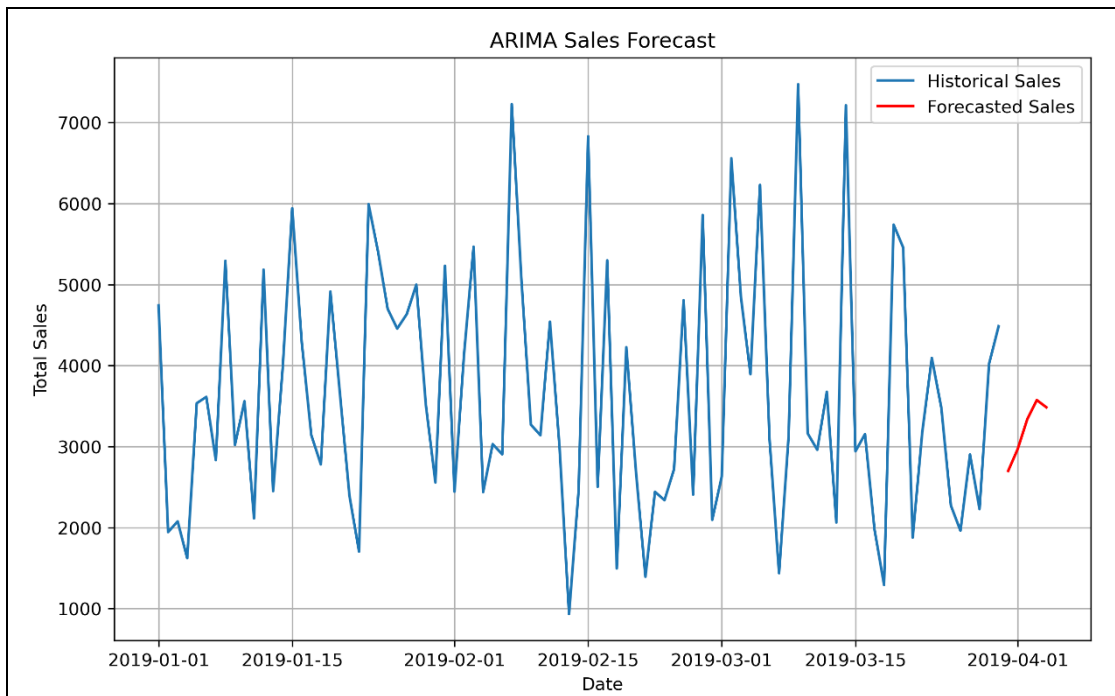
导入 CSV 文件后，显示前 5 行确认数据是否录入正常。

```

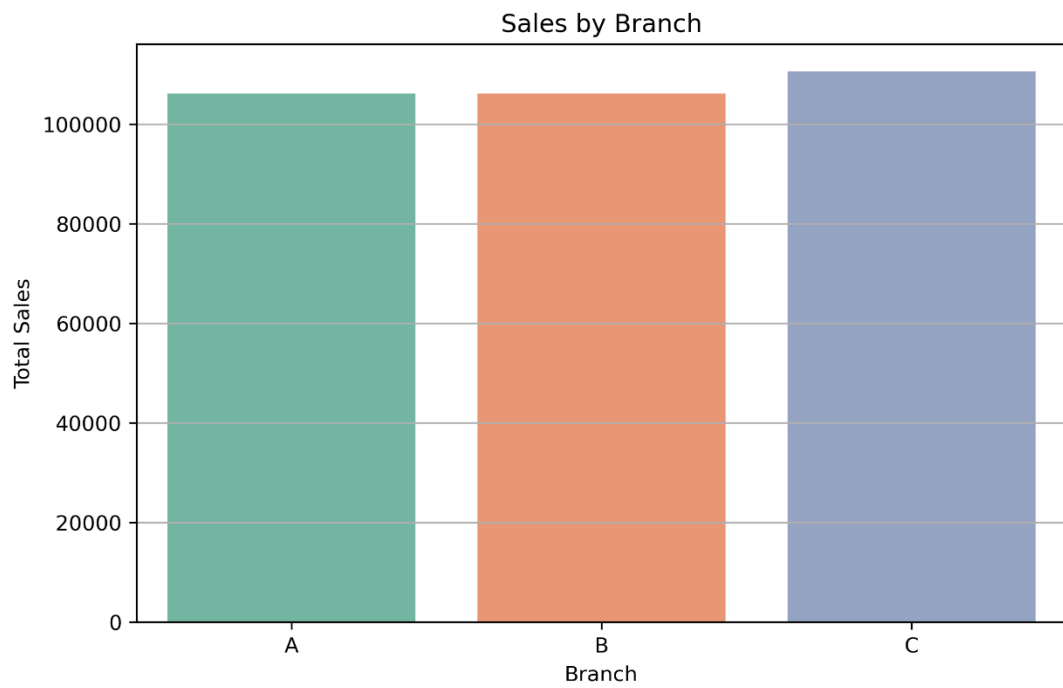
ARIMA模型预测未来5天的销售额：
2019-03-31    2700.616942
2019-04-01    2971.021460
2019-04-02    3338.935762
2019-04-03    3575.108956
2019-04-04    3485.929496
Freq: D, Name: predicted_mean, dtype: float64

```

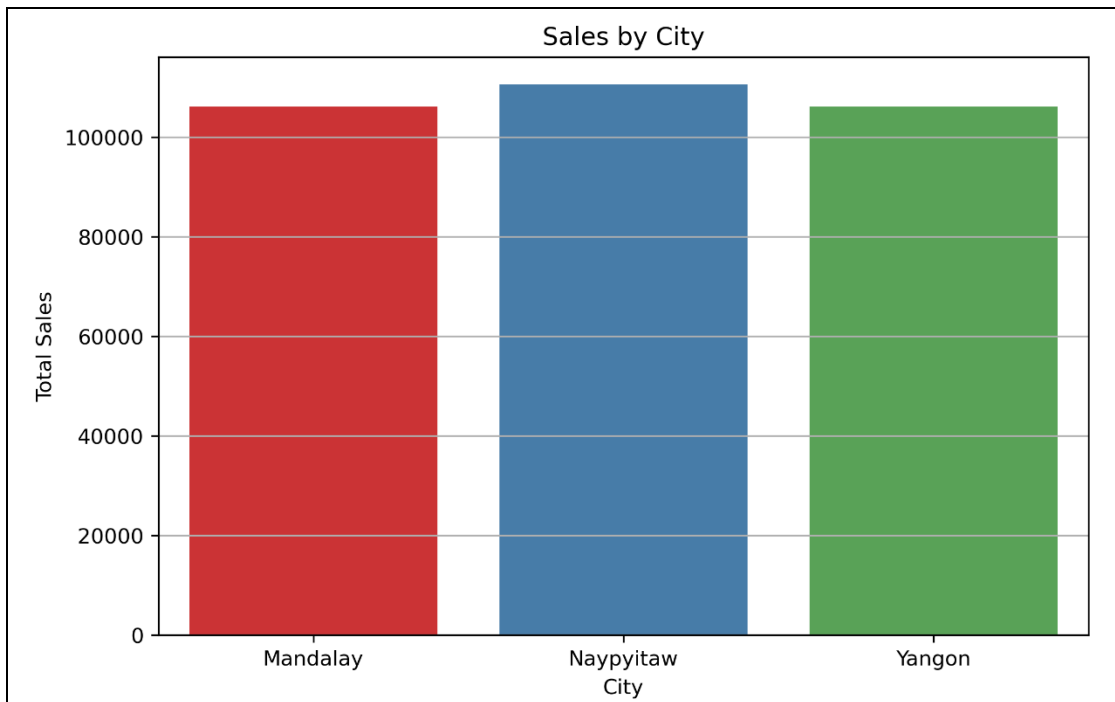
输出根据 ARIMA 模型预测未来 5 年的销售额的具体数值。



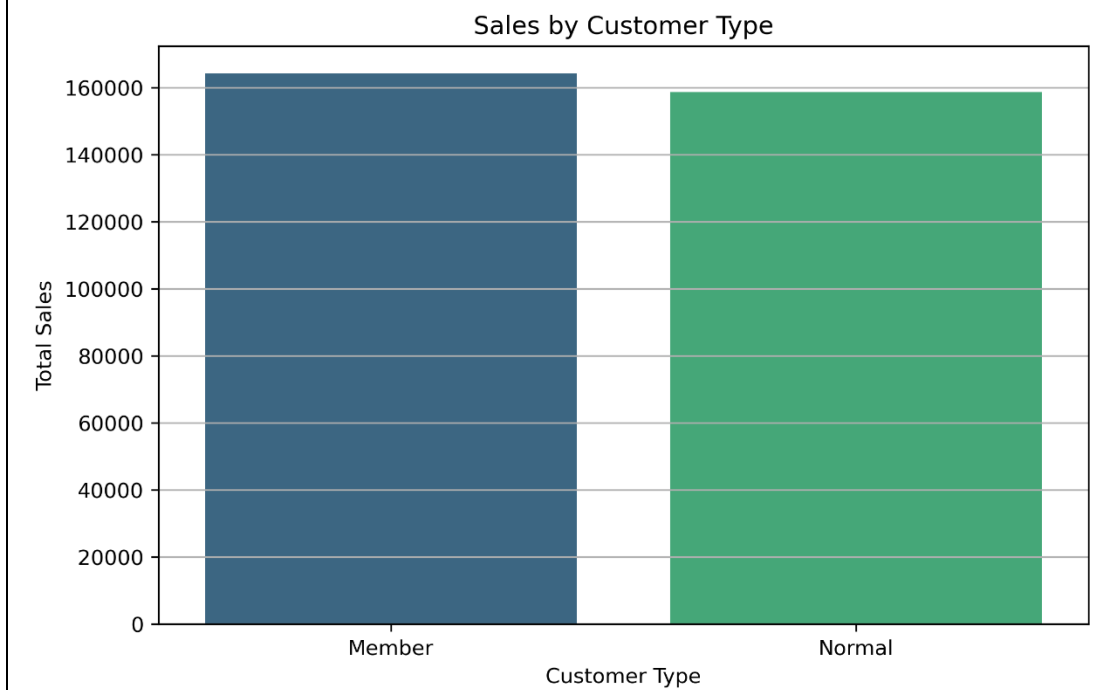
拟合 ARIMA 模型后给出的 5 天的销售额预测。



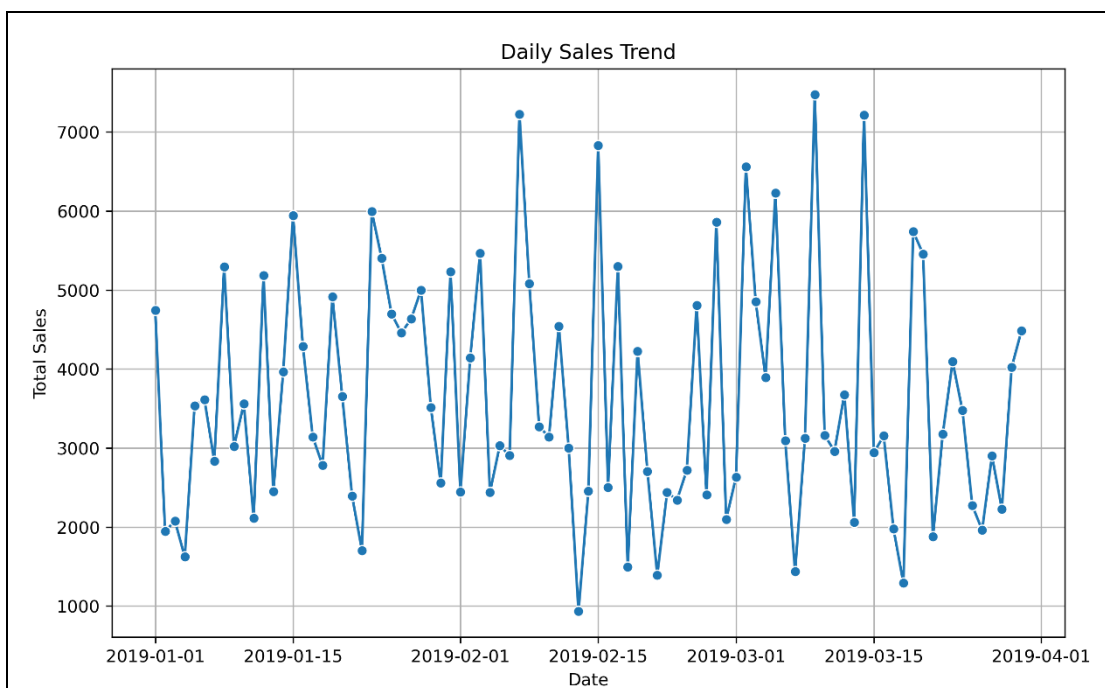
销售额在各分店的表现。



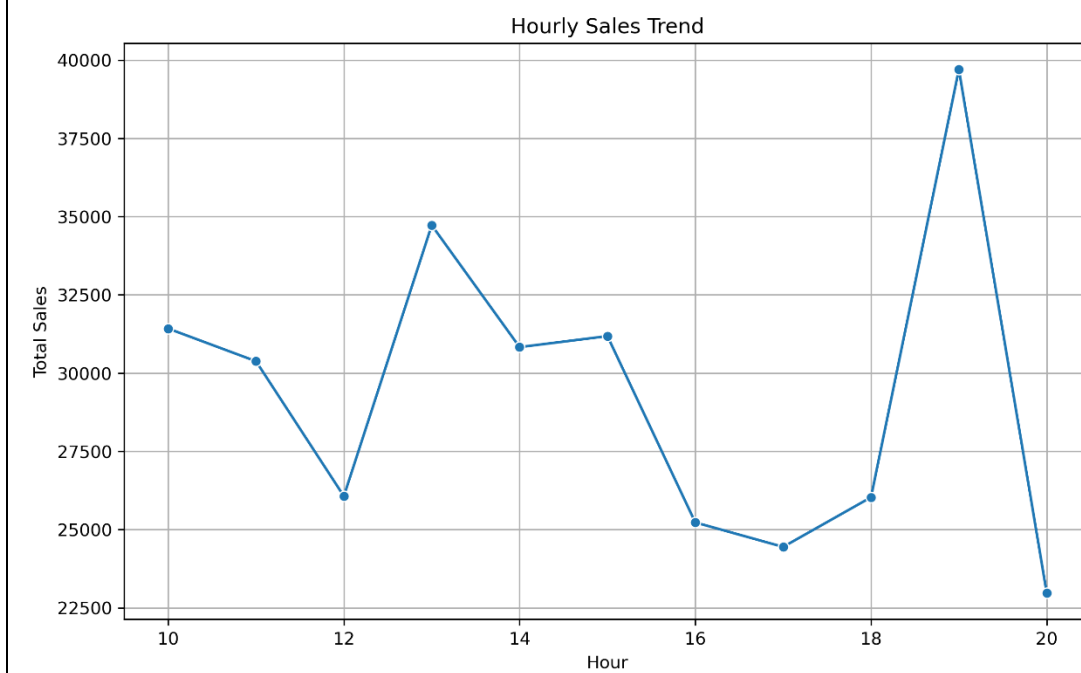
销售额在各分店的表现。



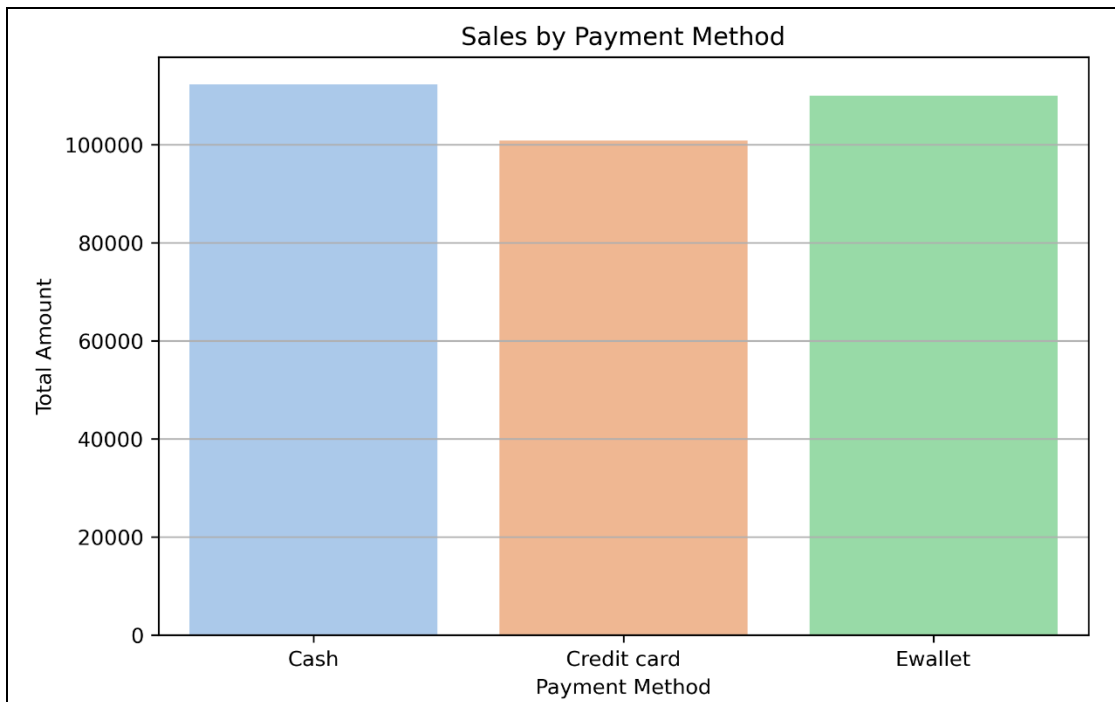
根据顾客类型分析的销售额。



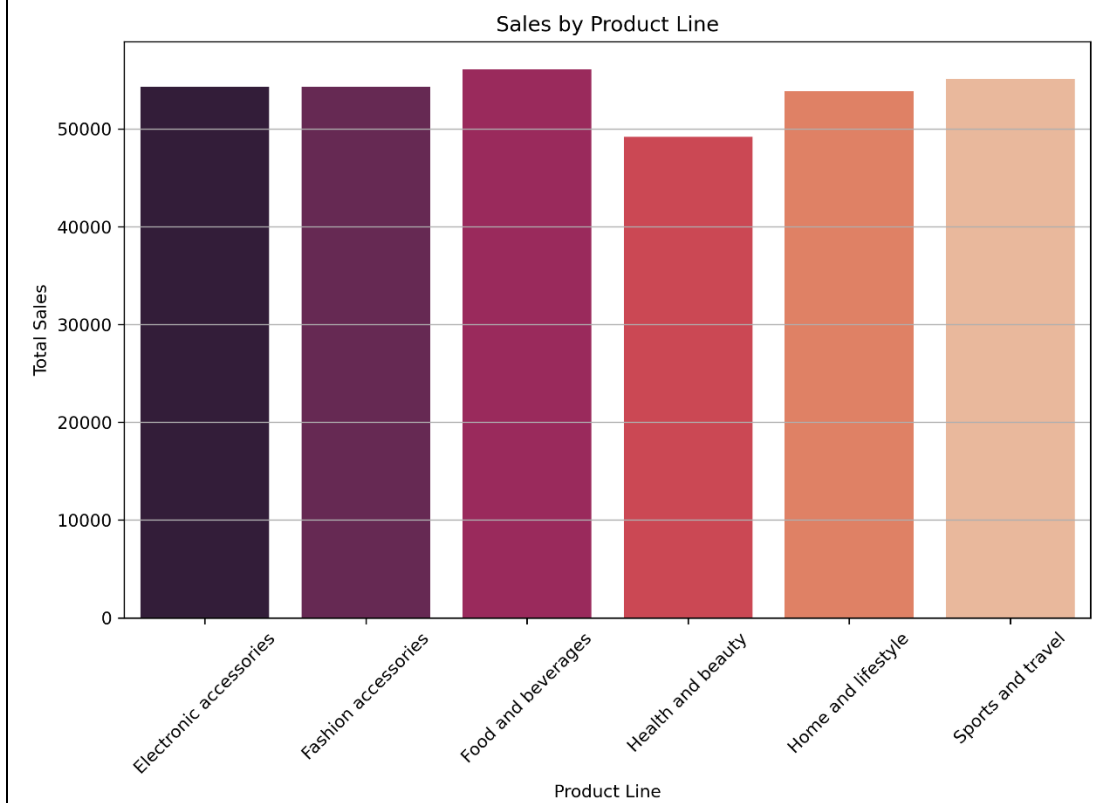
每日销售额趋势。



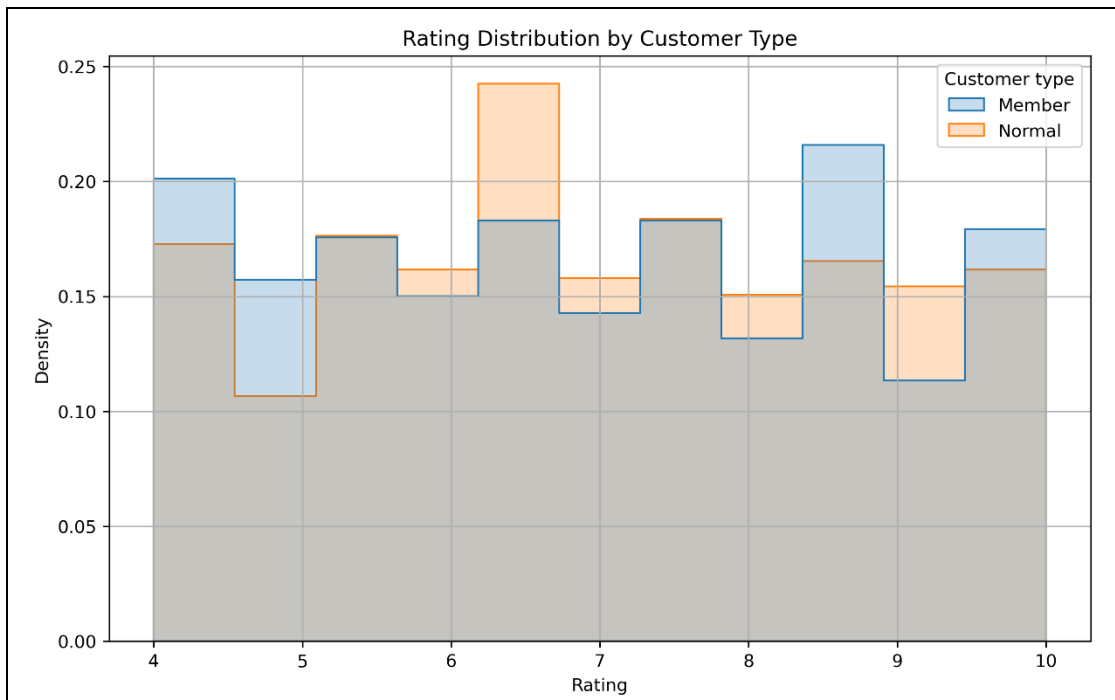
每小时销售额趋势。



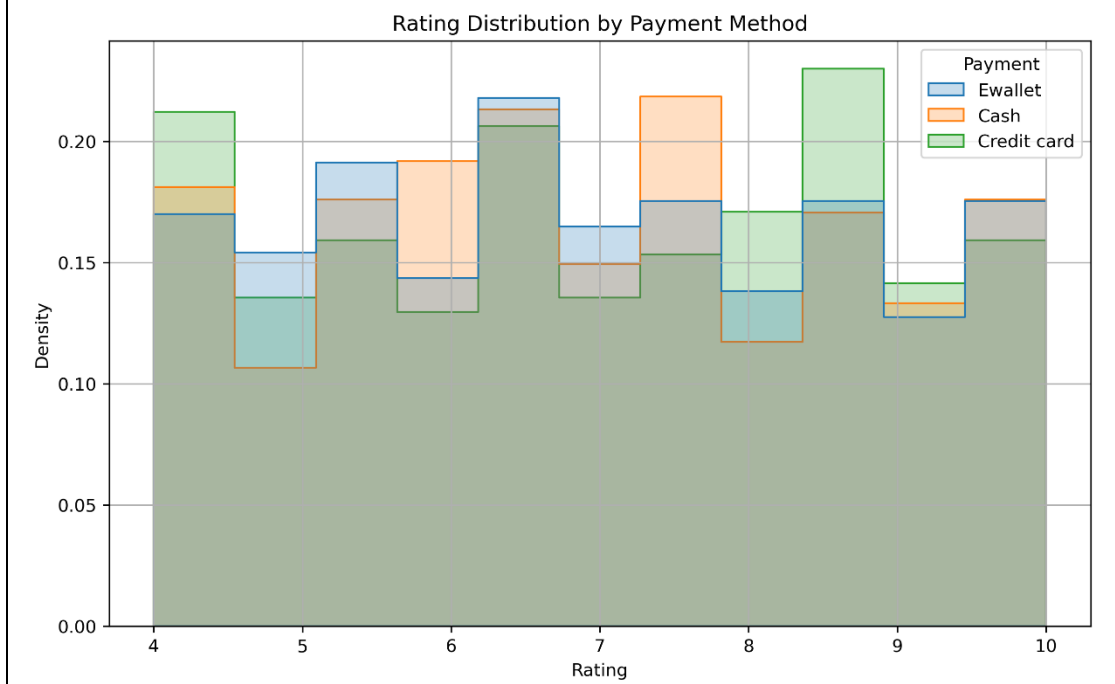
根据支付方式统计的销售额。



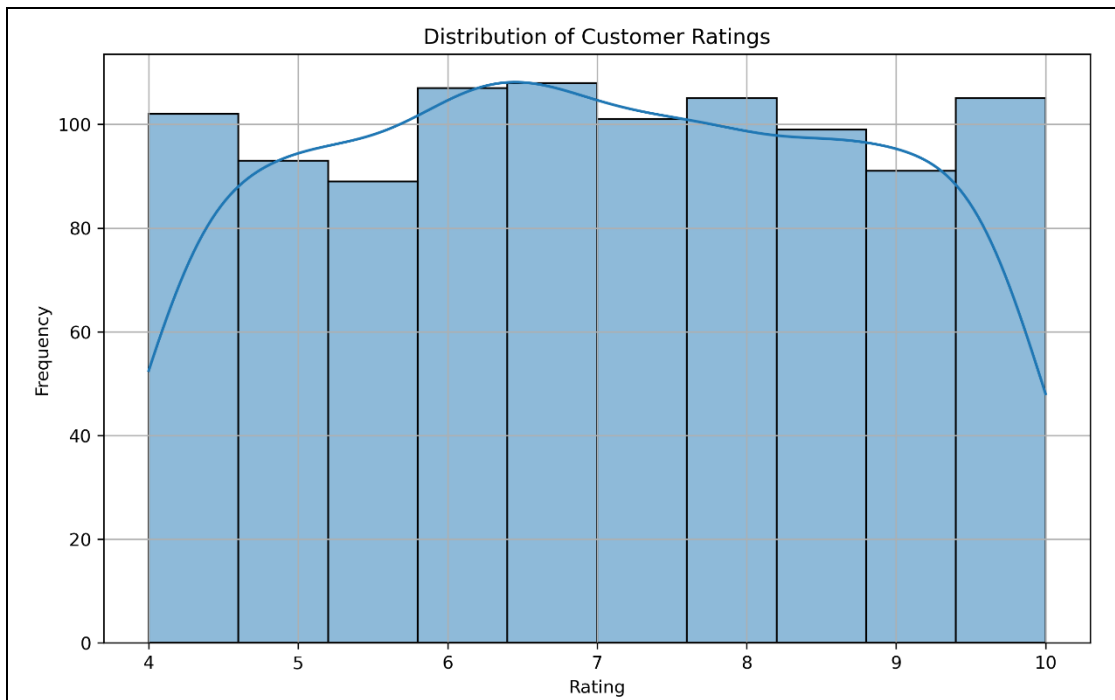
各产品线的销售额。



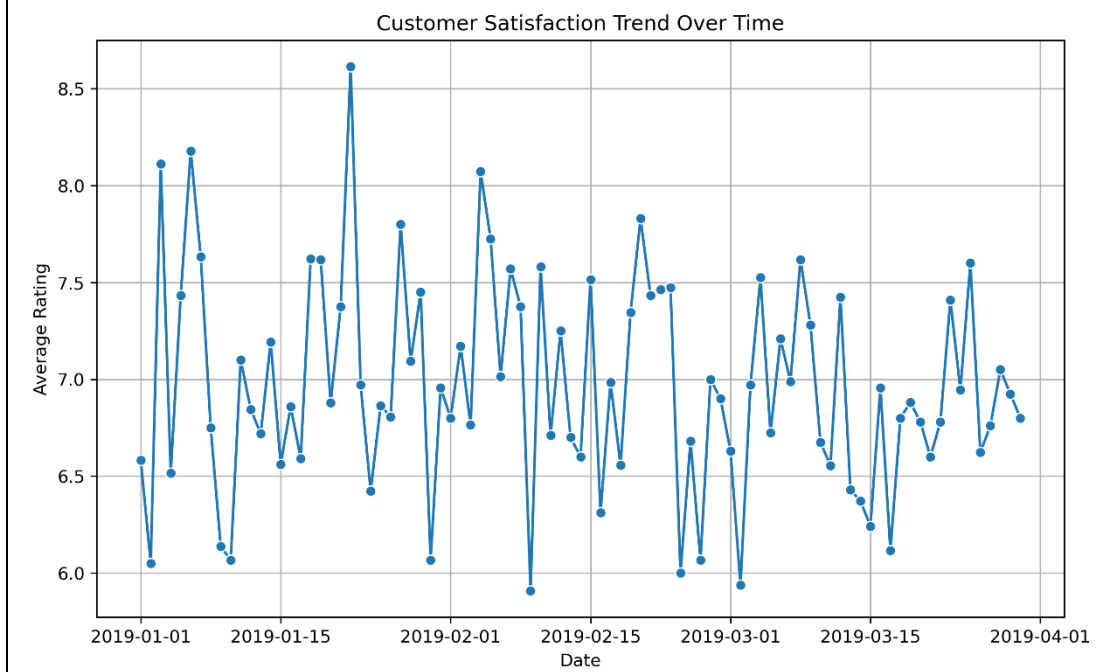
基于顾客类型分析的评分分布。



基于支付方式分析的评分分布。



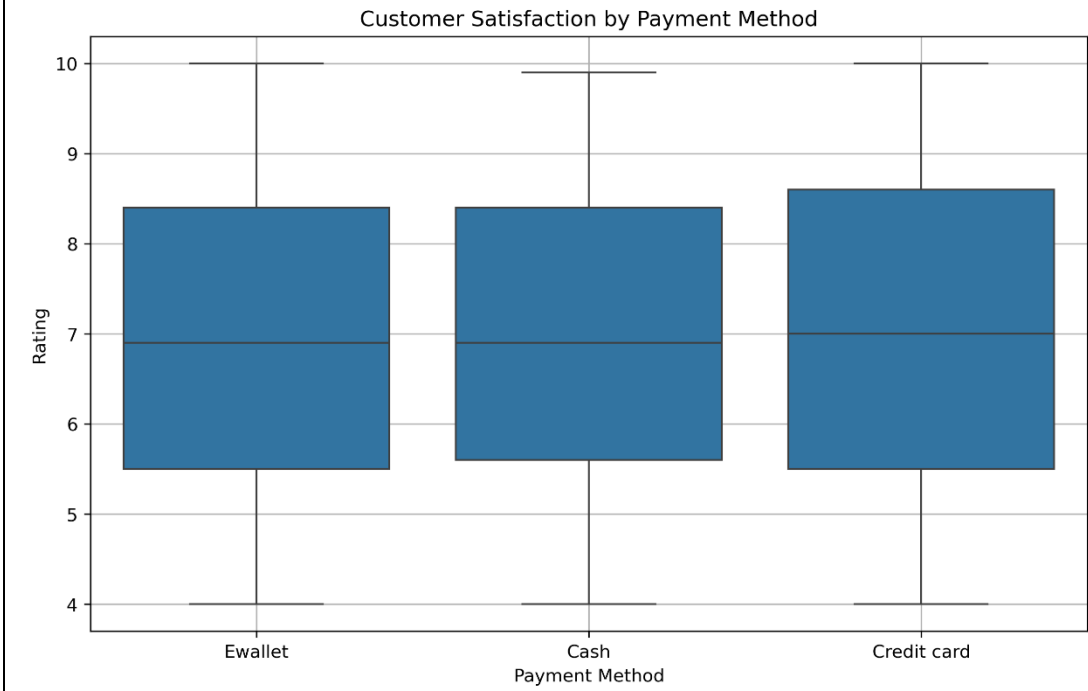
顾客评分分布。



顾客评分随时间的变动趋势。



基于顾客类型的顾客满意度分析。



基于支付方式的顾客满意度分析。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

注：成绩评定的内容可根据实际情况进行调整。